

# Operating Systems and System Programming

## Practical – 3

### Aim

To develop a C program using `fork()` to create a parent and child process, display their Process ID (PID), Parent Process ID (PPID), and process states at different stages, and design an experiment using Linux monitoring tools such as `ps`, `top`, and `/proc` to observe process state transitions between Ready, Running, Waiting, and Terminated.

### Algorithm:

1. Start the program.
2. Display the PID and PPID of the initial parent process.
3. Call `fork()` to create a child process.
4. Check the return value of `fork()`:
  - If it is negative, display an error.
  - If it is zero, execute the child process.
  - If it is positive, execute the parent process.
5. In the child process, display its PID and PPID and perform some operations with a delay to observe its state.
6. In the parent process, display its PID and PPID and wait for the child using `wait()`. 7. Use `ps` to observe the PID, PPID, and current process state.
8. Use `top` to continuously monitor the processes and observe their CPU activity and states.
9. Use `/proc/<PID>/status` or `/proc/<PID>/stat` to obtain detailed process-state information.
10. Observe the process during different stages:
  - Ready: process is waiting for CPU scheduling.
  - Running: process is currently executing.
  - Waiting: process is waiting for an event or resource.
  - Terminated: process has completed execution.
11. Record the observations from `ps`, `top`, and `/proc`.

12. After the child terminates, the parent continues and terminates.

13. Stop the program.

### Task-1:

#### Step-1:

```
allur@AKSHAYA:~$ nano practical3.c
```

#### Step-2:Program

```
GNU nano 8.7.1 practical3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
    }
}
```

```
GNU nano 8.7.1 practical3.c *
    printf("PPID = %d\n", getppid());

    printf("Child is running...\n");
    sleep(10);

    printf("Child is terminating...\n");
    exit(0);
}
else
{
    printf("Parent Process\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());
    printf("Child PID = %d\n", pid);

    printf("Parent is waiting...\n");
    wait(NULL);

    printf("Child terminated.\n");
}

return 0;
}
```

### Step-3: Compilation and Execution:

```
allur@AKSHAYA:~$ gcc practical3.c
allur@AKSHAYA:~$ ./a.out
Parent Process
PID      = 544
PPID     = 313
Child PID = 545
Parent is waiting...
Child Process
PID  = 545
PPID = 544
Child is running...
Child is terminating...
Child terminated.
allur@AKSHAYA:~$ |
```

---

### Task-2:

#### Commands

##### Step-1: Using ps:

```
allur@AKSHAYA:~$ ps -o pid,ppid,state,cmd
  PID   PPID  S  CMD
   313    311  S  -bash
   664    313  R  ps -o pid,ppid,state,cmd
allur@AKSHAYA:~$ |
```

##### Step-2: Using top:

```
allur@AKSHAYA:~$ top
top - 14:45:21 up 12 min, 1 user, load average: 0.01, 0.03, 0.00
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7651.2 total, 7028.7 free, 603.9 used, 170.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7047.3 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   24144  15340  11548 S   0.0   0.2   0:00.42 systemd
    2 root        20   0    3180   2200   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
    7 root        20   0    3180   2048   1940 S   0.0   0.0   0:00.00 init
   47 root        19  -1   50404  16784  15516 S   0.0   0.2   0:00.12 systemd-journal
   79 systemd+    20   0   22416  14436  11944 S   0.0   0.2   0:00.05 systemd-resolve
   85 root        20   0   35452  12400   9236 S   0.0   0.2   0:00.34 systemd-udevd
  105 root        20   0    2888   1948   1820 S   0.0   0.0   0:00.02 chronyd-starter
  106 root        20   0    4472   3128   2884 S   0.0   0.0   0:00.00 cron
  107 message+    20   0    8820   5396   4728 S   0.0   0.1   0:00.03 dbus-daemon
  113 root        20   0   44096  29744  14604 S   0.0   0.4   0:00.18 networkd-dispat
  140 root        20   0   18440   9424   8176 S   0.0   0.1   0:00.05 systemd-logind
  188 syslog      20   0  220548   5860   4616 S   0.0   0.1   0:00.05 rsyslogd
  210 root        20   0    5492   2708   2524 S   0.0   0.0   0:00.00agetty
  212 root        20   0  123460  32664  18052 S   0.0   0.4   0:00.17unattended-upgr
  214 _chrony      20   0   23600  11052   7124 S   0.0   0.1   0:00.05 chronyd
  232 _chrony      20   0   12096   2404   1760 S   0.0   0.0   0:00.00 chronyd
  310 root        20   0    3184   1104    980 S   0.0   0.0   0:00.00 SessionLeader
  311 root        20   0    3200   1244   1108 S   0.0   0.0   0:00.05 Relay(313)
  313 allur       20   0    6268   5540   3864 S   0.0   0.1   0:00.05 bash
  314 root        20   0    8644   5524   4684 S   0.0   0.1   0:00.00 login
  376 allur      20   0   22340  13548  10936 S   0.0   0.2   0:00.07 systemd
  378 allur      20   0   22920   4072   2068 S   0.0   0.1   0:00.00 (sd-pam)
  404 allur      20   0    6136   5476   3924 S   0.0   0.1   0:00.01 bash
  674 allur      20   0    8144   6040   3900 R   0.0   0.1   0:00.00 top
```

### Step-3: Using /proc:

```
allur@AKSHAYA:~$ ./a.out
Parent Process
PID      = 919
PPID     = 313
Child PID = 920
Parent is waiting...
Child Process
PID      = 920
PPID     = 919
Child is running...
cat /proc/904/status
cat /proc/920/status
Child is terminating...
Child terminated.
```



**Result**

The experiment successfully demonstrated the creation and execution of parent and child processes using `fork()`. The PID, PPID, and process states were observed at different stages using `ps`, `top`, and `/proc`. The experiment showed how a process transitions through Ready, Running, Waiting, and Terminated states during its lifetime, providing a practical understanding of Linux process management and scheduling.